

Concord Lite

AG2 Workflow Under Test -> Contract Violation -> Repair

Updated 5-page hackathon build document

1. North Star

One-line definition

Concord Lite is a contract-to-repair layer for multi-agent workflows. It reads an AG2 run trace plus a workflow contract, detects violated role, state, tool, routing, and approval contracts, recommends an AG2-native repair patch, and generates a regression test so the same failure does not return.

Why this can win

- It is a framework-level build, not just another app.
- It uses AG2 terms correctly and visibly: GroupChat, GroupChatManager or AutoPattern, ContextVariables, Handoffs, Guardrails, UserProxyAgent.
- It gives judges a concrete outcome: violated contract, repair patch, and regression test.
- It visibly uses sponsors in the product path, not just in the slide deck.

Prize fit

- Primary: **Best Multi-Agent Collaboration**
- Secondary: **Most Impressive AG2 Build**
- Optional research angle: use a literature review workflow under test and pitch it under **Scientific Research**

Scope discipline

We are **not** rebuilding AG2 Failure Attribution. That tells who failed and when. Concord Lite tells what contract broke, what primitive should change, and what regression test prevents recurrence.

Concord Lite Architecture: AG2 Workflow Under Test → Contract Violation → Repair

A correctness layer for multi-agent workflows built with AG2, Tavily, and Daytona

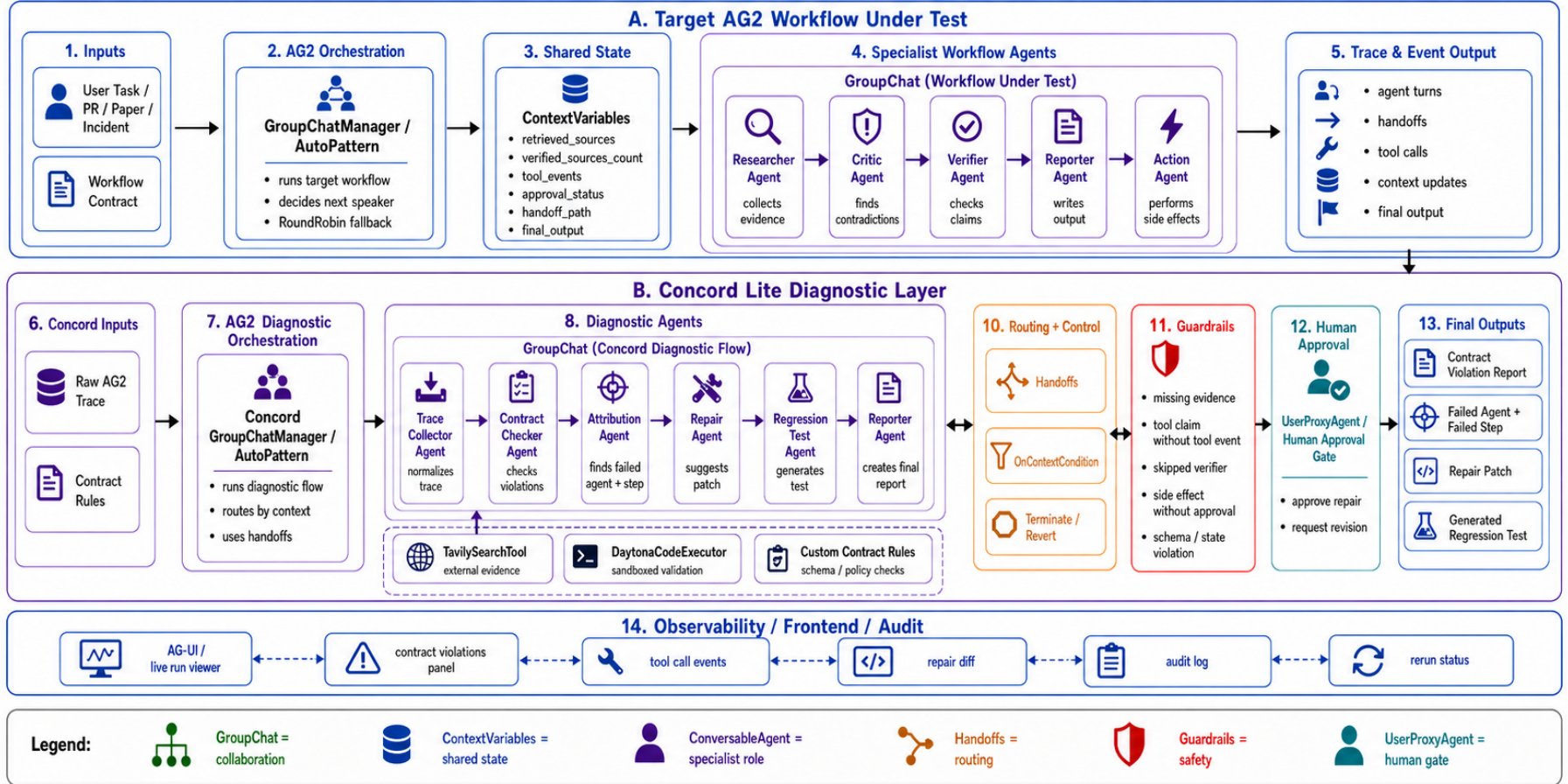


Figure 1: Concord Lite architecture

2. End-to-End Flow and Product Boundary

Two-zone architecture

- **Zone A** is the target AG2 workflow under test.
- **Zone B** is the Concord Lite diagnostic layer.
- Keeping these separate is the main architectural insight.

End-to-end flowchart

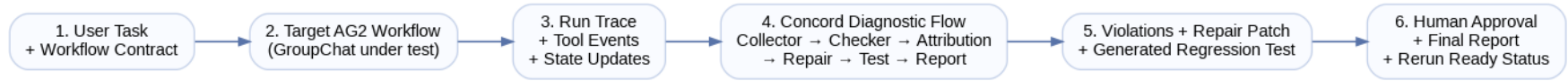


Figure 2: End-to-end flow

Product boundary

Layer	Purpose	Main AG2 pieces	Output
Target workflow under test	Produce the original artifact such as a literature review, PR review, incident memo, or lab summary	GroupChatManager or AutoPattern, Researcher, Critic, Verifier, Reporter, ActionAgent, ContextVariables, Handoffs	Final output plus run trace, tool events, state updates
Concord diagnostic layer	Inspect the run rather than solve the original task	Separate GroupChat with Trace Collector, Contract Checker, Attribution, Repair, Regression Test, Reporter, UserProxyAgent	Violations, failed agent plus step, repair patch, regression test, final report

Contract checks we must support

Contract type	Example rule	Failure Concord catches	Repair pattern
Evidence contract	Reporter requires verified sources	Final answer written with zero verified sources	Route back to Researcher and add citation guardrail
Tool contract	Search or execution claims require a matching tool event	Verifier claims it checked evidence, but no Tavily or Daytona event exists	Require tool event before verdict
Routing contract	Reporter must run after Verifier	Workflow bypasses Verifier	Add OnContextCondition or explicit Handoff
Approval contract	ActionAgent requires approval status = approved	Side effect reachable before UserProxyAgent	Insert HumanGate
Schema contract	Final report must match schema	Missing citations or repair fields	Add schema guardrail and regenerate

Sponsor usage plan

Sponsor asset	How we use it	Why it matters
\$100 Daytona credit	Run generated regression test or sandboxed validation	Makes verification visible and real
\$30 V0 credit	Rapidly ship polished frontend screens	Helps the demo look premium fast
AG2 platform credit	Power both AG2 flows	Strong sponsor alignment
Gemini key or credit	Backup model path for explanation or analysis	Adds resilience if the primary path fails

3. Component Inputs, Processing, and Outputs

Diagnostic components

Component	Input	Processing responsibility	Output
Trace Collector Agent	Raw AG2 trace, messages, tool events, context deltas	Normalizes the run into one ordered timeline	normalized_trace, handoff_path, tool_events, context_snapshots
Contract Checker Agent	normalized_trace plus contract rules	Runs deterministic checks first, then LLM explanation	violations[] with type, severity, expected, observed, evidence
Attribution Agent	violations plus trace	Identifies failed agent and decisive step as supporting metadata	failed_agent, failed_step, likely_root_cause
Repair Agent	violation plus failed primitive	Maps failure to AG2 repair pattern	repair_patch, affected_primitive, expected_impact
Regression Test Agent	repair_patch plus violation	Generates a test and runs or simulates it in Daytona	test_name, assertions, test_code, test_status
Reporter Agent	all diagnostic outputs	Builds final operator and judge-facing artifact	Contract Violation Report
UserProxyAgent or HumanGate	repair proposal plus report	Approves, rejects, or requests revision	approval_status, comments

Core objects

Object	Required fields
RunTrace	run_id, workflow_name, events[], final_output
TraceEvent	step, timestamp, agent, type, content, tool_call_id, context_delta, handoff_to
ContextSnapshot	retrieved_sources, verified_sources_count, tool_events, approval_status, failed_agent, failed_step
ToolEvent	tool_name, input, output, status, evidence_id, timestamp
RoleContract	agent_name, allowed_actions, required_inputs, forbidden_outputs
EvidenceContract	claim_type, required_source_count, required_tool_event
RouteContract	from_agent, to_agent, condition, forbidden_path
ApprovalContract	side_effect, required_approval_state, human_gate

API endpoints

Endpoint	Purpose	Returns
/api/demo-run	Loads curated broken AG2 run	RunTrace and ContextVariables snapshot
/api/analyze	Runs Concord checks and diagnostic flow	violations[], failed_agent, failed_step
/api/repair	Maps violation to AG2 primitive repair	repair_patch and affected_primitive
/api/test	Runs or simulates regression test in Daytona	test_status, stdout, assertions
/api/report	Builds final artifact	Contract Violation Report JSON

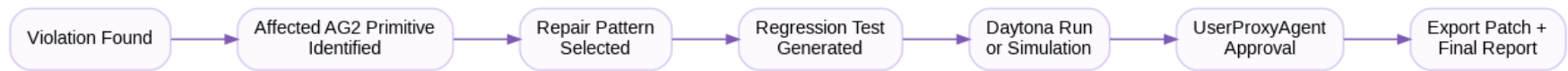


Figure 3: Failure to repair flow

Failure to repair flowchart

MVP truth

Backend can be partially simulated if the UI is polished and the contract-to-repair loop is honest. Deterministic checks keep the demo stable. AG2 agents make the reasoning and repair narrative feel intelligent.

4. What We Build on Hackathon Day

Must-have build artifacts

Artifact	Owner	Definition of done
Polished frontend dashboard	P + N	6 screens: Live Run, Workflow Detail, Violation Detail, Repair, Regression Test, Final Report
Demo trace plus contract JSON	A + D	One broken run with 4 violations: missing evidence, skipped verifier, tool claim without tool event, side effect without approval
AG2 diagnostic GroupChat	A	Collector -> Checker -> Attribution -> Repair -> Test -> Reporter
Contract checker API	D	/analyze returns violations, repair_patch, regression_test, report
Tavily integration	D	Used for evidence-search check or convincingly mocked with captured response
Daytona integration	D	Runs or simulates test execution visibly in the UI
Demo script plus backup video	N	4-minute rehearsed path with no dead air

Team split

Person	Lane	Exact responsibilities
A	Backend and AG2 core	Define AG2 agents, GroupChat, AutoPattern or fallback, trace schema, prompts, mocked target workflow
D	Backend and tools	FastAPI endpoints, checker rules, Tavily path, Daytona path, regression runner, API payloads
P	Frontend and product UI	V0 or React dashboard, timeline, tabs, cards, repair diff, final report, loading states
N	Design and demo	Design system, icons, motion, copy, pitch, sample data polish, narration, Q and A prep

Hour-by-hour plan

Time	A + D backend	P + N frontend and design	Checkpoint
11:00–11:30	Lock schema, endpoints, agents, demo trace	Create UI shell, theme, routes	Trace renders
11:30–12:30	Implement /analyze deterministic checks	Dashboard, workflow graph, violation cards	Violations visible
12:30–1:30	Diagnostic agents produce explanation and report	Trace timeline and violation detail	Agent trace looks alive

Time	A + D backend	P + N frontend and design	Checkpoint
1:30–2:30	Tavily and Daytona hooks or recorded responses	Repair diff and regression test screen	Sponsor tools visible
2:30–3:15	Stabilize outputs and fallbacks	Final report and approval screen	End-to-end demo works
3:15–4:00	Freeze code and record fallback	Polish and rehearse	No new features

Important execution note

A and D must keep the backend truthful but limited. P and N must make the UI feel like a real AI ops product. In hackathons, visual clarity wins attention first.

5. Demo, Evaluation Fit, Risks, and Important Notes

Demo flowchart



Figure 4: Demo flow

4-minute demo path

Time	What judges see	Line to say
0:00–0:30	Dashboard opens with a failed Literature Review Assistant run	This AG2 workflow gave a confident answer, but violated its evidence contract.
0:30–1:15	Target workflow trace: Researcher, Critic, Verifier, Reporter, ActionAgent	We are not asking a chatbot. We are inspecting a multi-agent run.
1:15–2:00	Concord diagnostic GroupChat runs and violations appear	The contract checker found missing evidence, a tool claim without tool event, and an unsafe action path.
2:00–2:45	Repair screen shows AG2 primitive affected	The fix is not vague: add this ContextVariables check, this Handoff, this Guardrail, this HumanGate.
2:45–3:30	Daytona runs generated regression test and Tavily evidence check is visible	Sponsors are not stickers. They are part of verification.
3:30–4:00	Final report plus approval and rerun-ready status	AG2 builds agents. Concord makes their workflows correctible.

Track mapping

Prize track	How Concord Lite fits
Best Multi-Agent Collaboration	Two AG2 GroupChats with clear role separation. Six diagnostic agents handle separate responsibilities.
Most Impressive AG2 Build	It is a framework-level product that turns traces into violations, repair patches, and tests.
Scientific Research	If the workflow under test is a literature-review assistant, Concord improves research reliability by blocking unsupported claims.

Acceptance criteria

- First screen explains the product in under 10 seconds.
- At least 4 contract violations are detected from one trace.
- Repair patch names a specific AG2 primitive.

- Generated regression test is shown and marked passed or simulated via Daytona.
- Final report is export-ready and visually polished.

Risks and mitigations

Risk	Mitigation
Backend incomplete	Use deterministic rules and fixture payloads
API or model instability	Keep static fallback JSON and captured tool responses
UI looks weak	Prioritize V0 polish early
Judges think this is only attribution	Repeat contract -> repair -> test framing

Questions judges may ask

Question	Answer
Is this just AG2 Failure Attribution?	No. Attribution is supporting metadata. The core product is violated contract, affected primitive, repair patch, and regression test.
What is live versus mocked?	Live UI, checker flow, report, and most payload logic. Tavily or Daytona can fall back to captured responses.
Why multi-agent?	Trace normalization, checking, attribution, repair, testing, and reporting are different responsibilities and benefit from separate roles.
What is the long-term product?	A framework-agnostic correctness layer for AG2, LangGraph, CrewAI, OpenAI Agents SDK, Google ADK, A2A, and MCP-based workflows.

Final note for the team

The source-of-truth mental model is always the same:

workflow under test -> contract violation -> repair -> regression test

That is the line every teammate should know and every judge should remember.